EXPERIMENT-08

Aim

To design and implement an automated cloud-based backup and monitoring system on AWS that detects newly created files on an EC2 instance, uploads them to Amazon S3, sends email notifications through Amazon SNS, and performs centralized log monitoring using Amazon CloudWatch, with access controlled through IAM.

Software / Tools Required

- Amazon Web Services (AWS) account with EC2, S3, SNS, CloudWatch, and IAM access
- Amazon EC2 instance (Linux)
- Python 3 with Watchdog and Boto3 libraries
- AWS CloudWatch Agent
- Text editor / terminal (SSH client)

Theory

Manual backup of files is time-consuming and prone to human error. Cloud platforms such as AWS provide services that can be combined to automate this process. Amazon EC2 hosts the monitoring application; the Watchdog library observes a local folder for filesystem events; Boto3, the AWS SDK for Python, uploads detected files to an Amazon S3 bucket for durable storage; Amazon SNS publishes a notification to a topic, which forwards an email to a subscribed address confirming the backup; and Amazon CloudWatch, through the CloudWatch Agent, streams a local application log file to a centralized CloudWatch Log Group for remote monitoring. An IAM role attached to the EC2 instance grants the application the minimum permissions needed to call S3, SNS, and CloudWatch without embedding access keys in code, while separate IAM users (Admin and Viewer) demonstrate role-based access control.

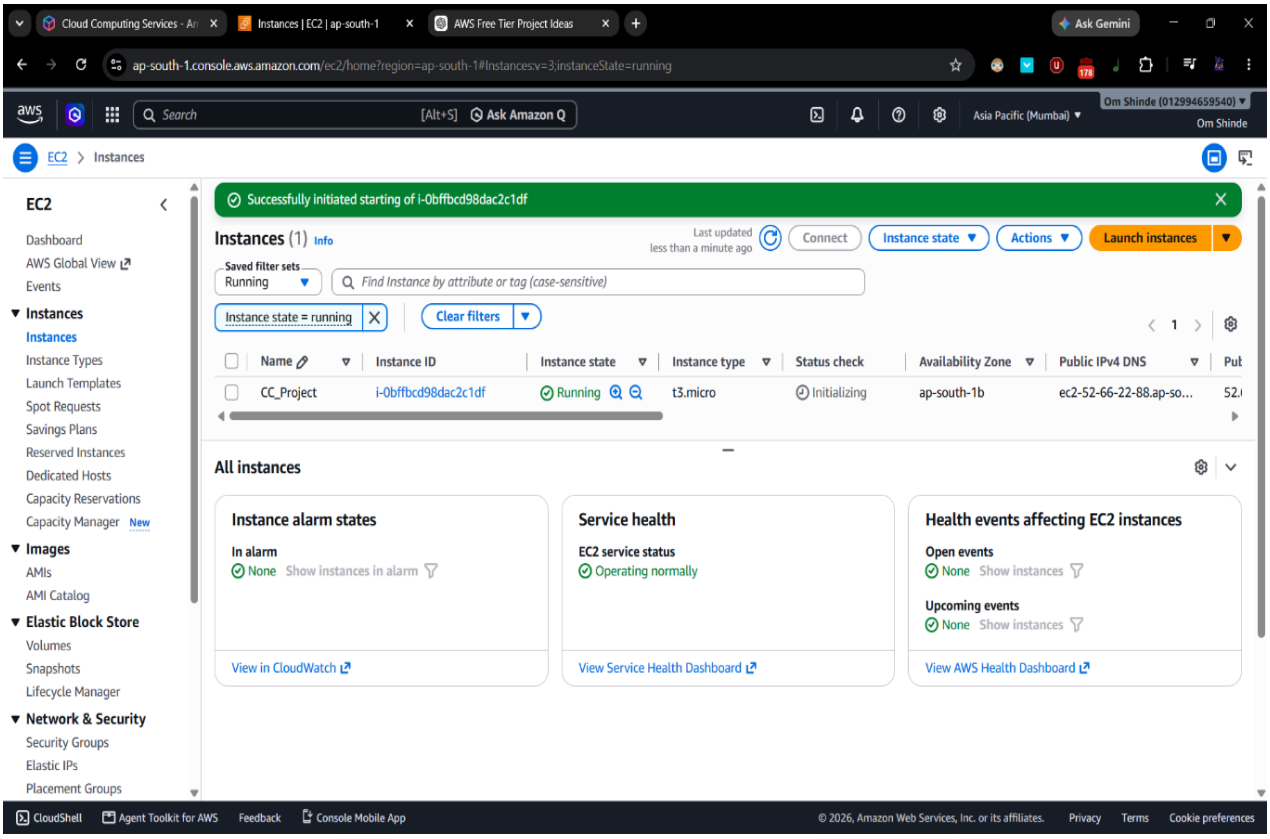
System Architecture / Working

Project Folder → Watchdog (file-creation event) → Python Application (Project.py) → Boto3 → Amazon S3 bucket 'abcnewaws' (backup) → Amazon SNS topic 'ProjectNotification' → Email notification. In parallel, all events are written to 'ProjectLogs.log' → CloudWatch Agent → CloudWatch Log Group 'ProjectLogs' for centralized monitoring. IAM Role (attached to EC2) authorizes all AWS API calls; IAM Users (Admin, Viewer) control console-level access.

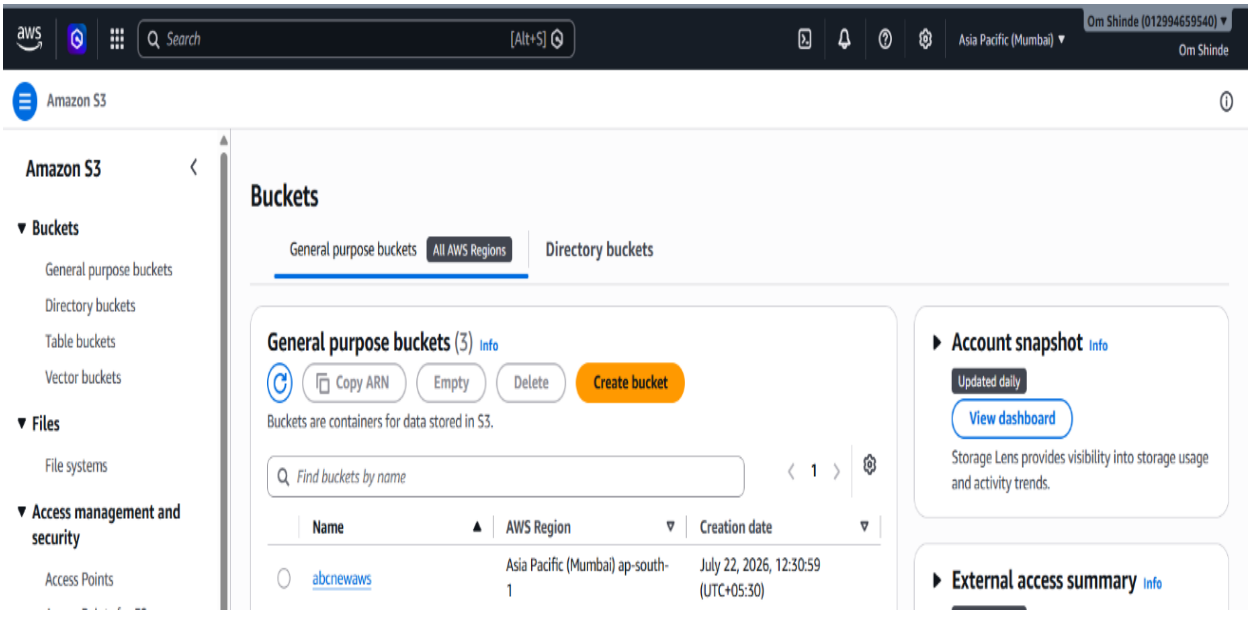
Procedure

1. Launch an Amazon EC2 instance and create an IAM role with permissions for S3, SNS, and CloudWatch; attach the role to the instance.
2. Create an S3 bucket named 'abcnewaws' to store backed-up files.
3. Create an SNS topic named 'ProjectNotification' and subscribe an email address to it; confirm the subscription.
4. On the EC2 instance, create a folder named 'ProjectFolder' to be monitored, and install Python, Boto3, and Watchdog.
5. Write 'Project.py' to: (a) watch ProjectFolder using Watchdog, (b) upload any newly created file to the S3 bucket using Boto3, (c) publish a message to the SNS topic after a successful upload, and (d) log every activity to 'ProjectLogs.log'.
6. Install and configure the CloudWatch Agent on the EC2 instance to monitor 'ProjectLogs.log' and stream it to the CloudWatch log group 'ProjectLogs'.
7. Create IAM users 'Admin' and 'Viewer' with differing permission policies to demonstrate access control.
8. Run 'Project.py' and add a test file to ProjectFolder to verify the complete flow: detection → upload → notification → logging → CloudWatch monitoring.

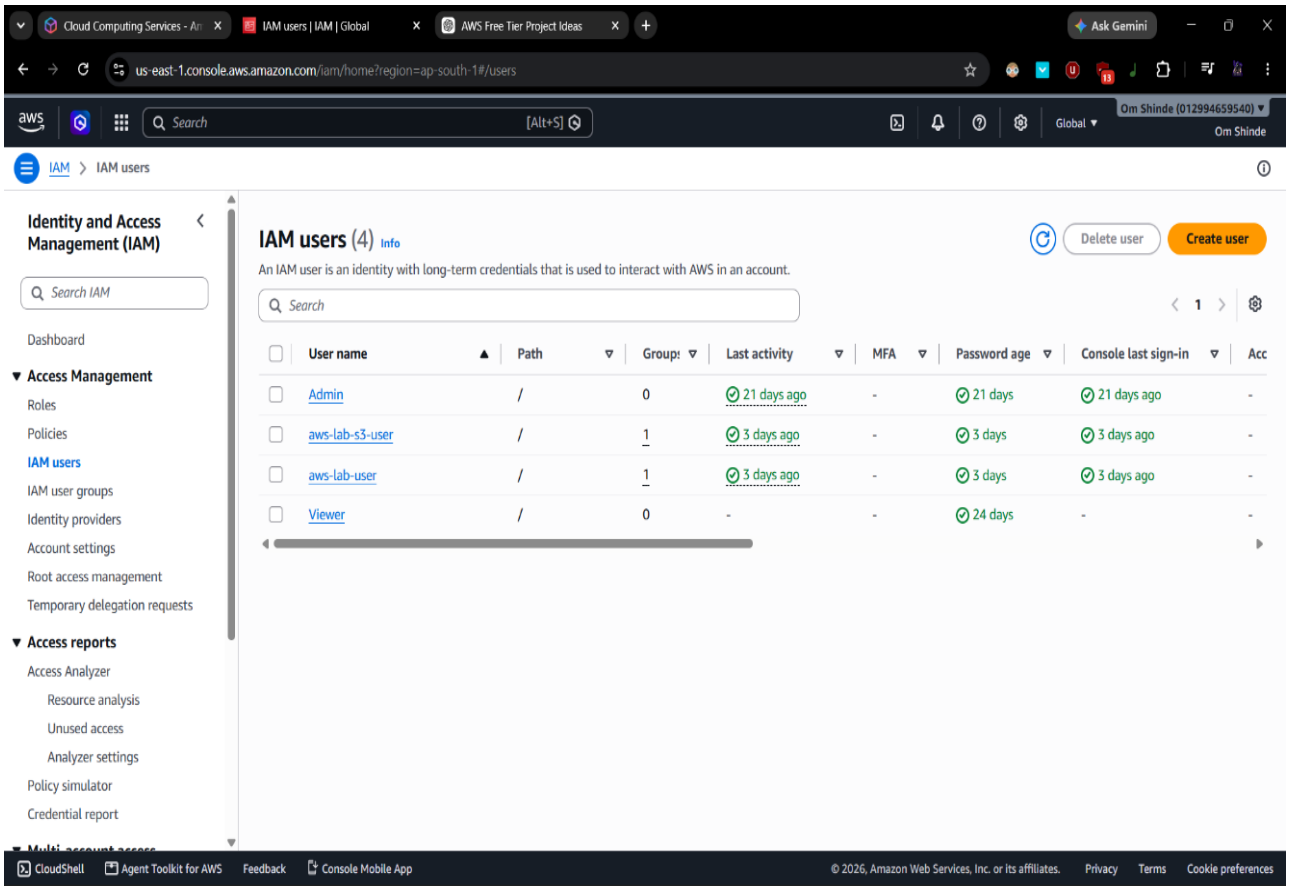
Output



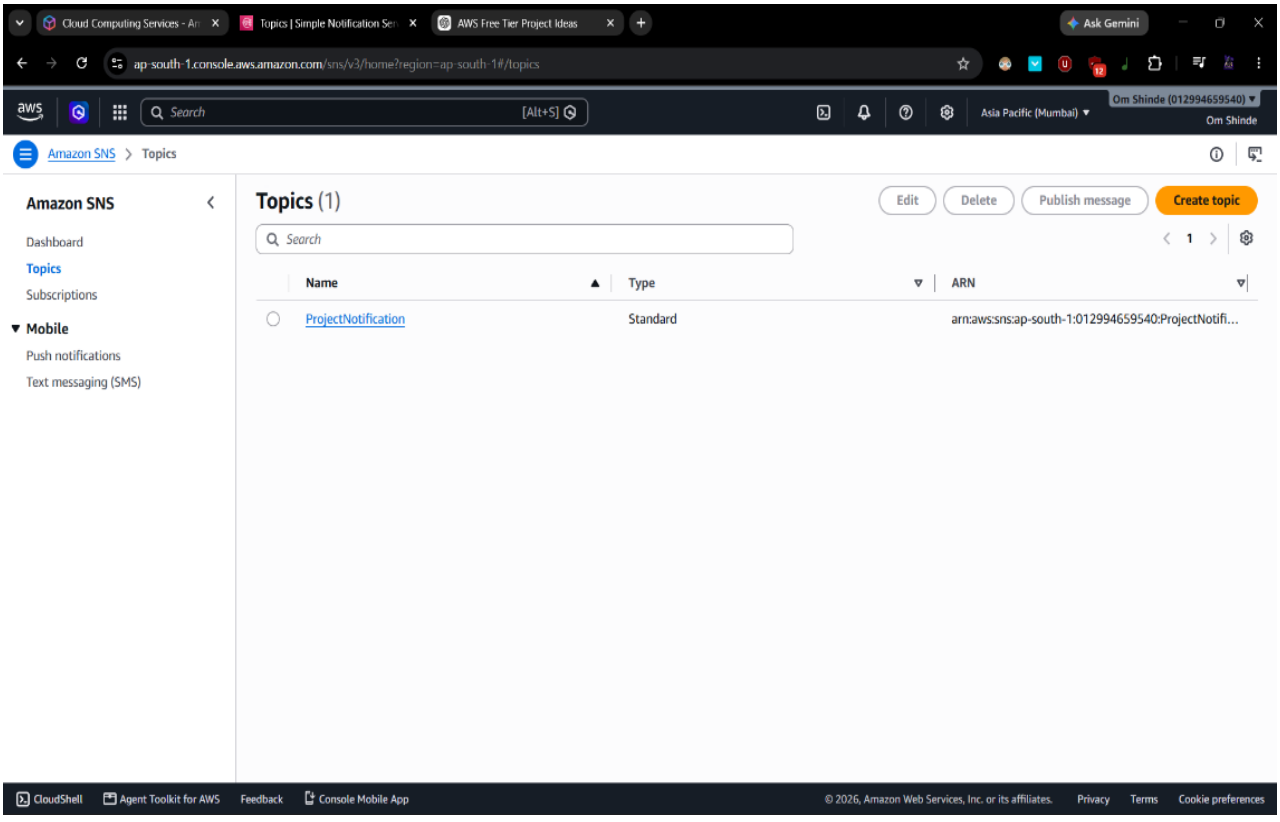
EC2 Running



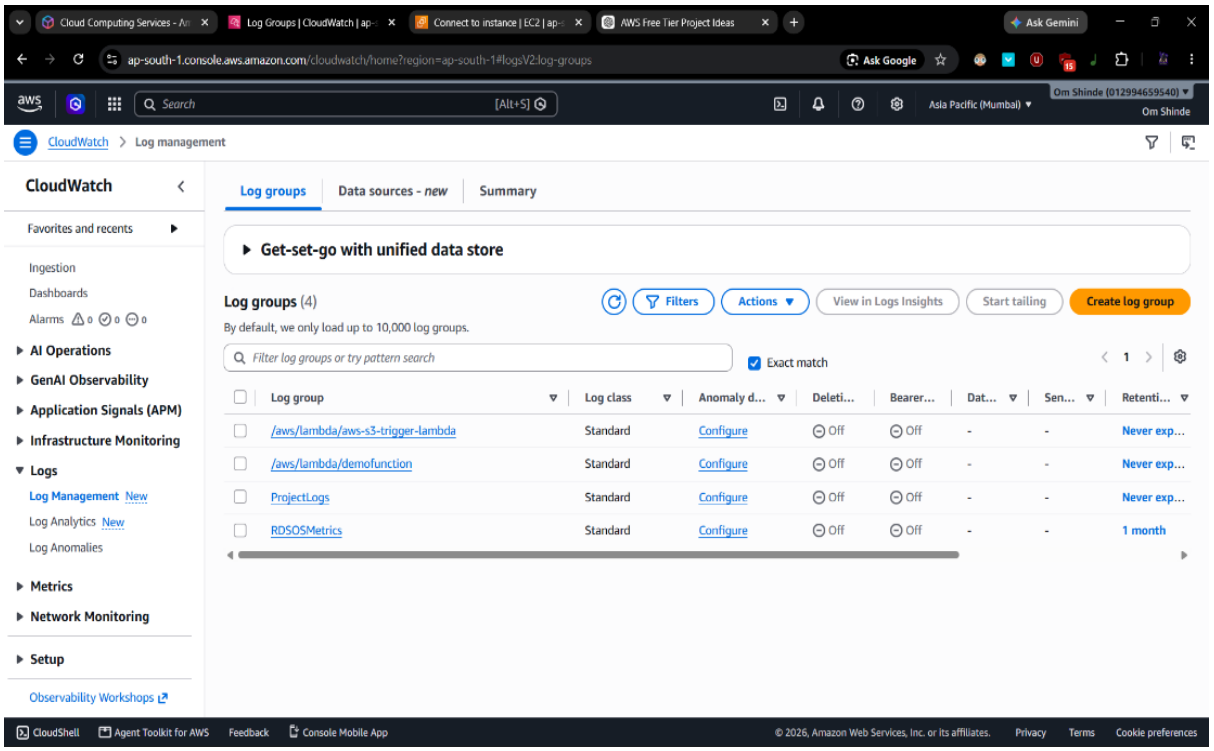
S3



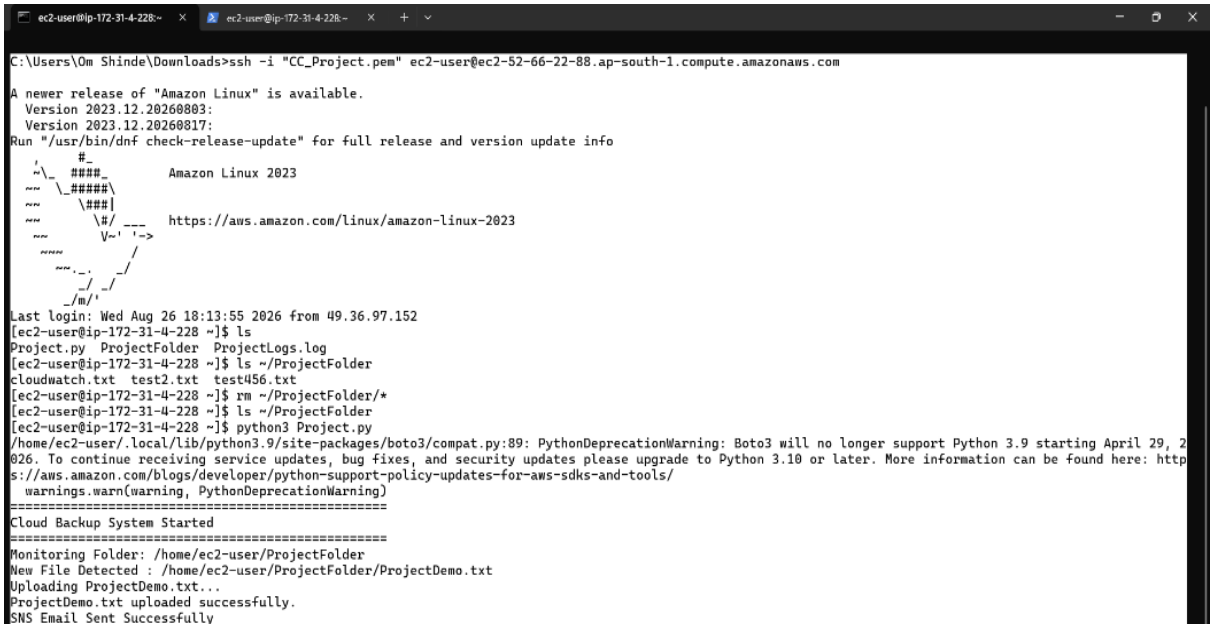
IAM Users

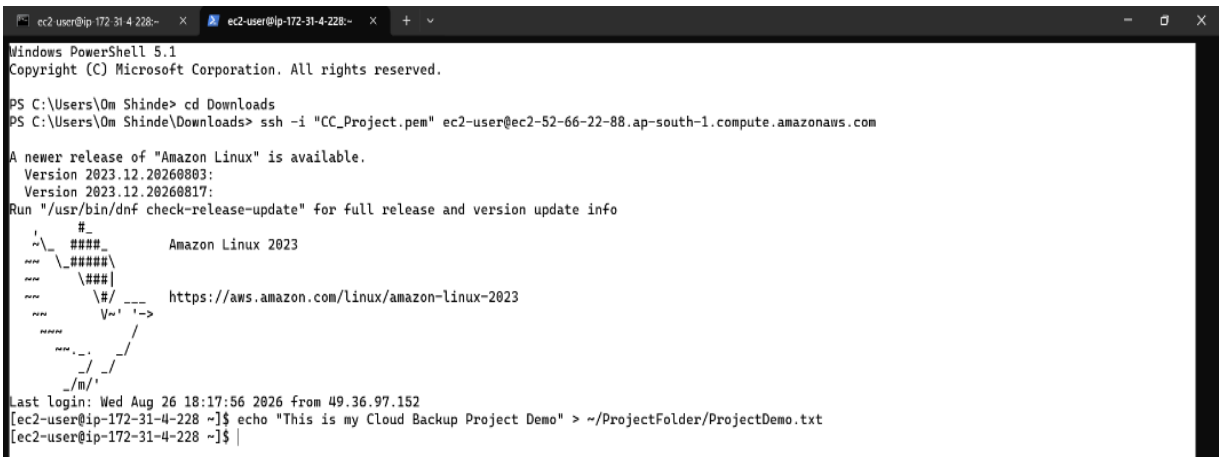


SNS Topic Active

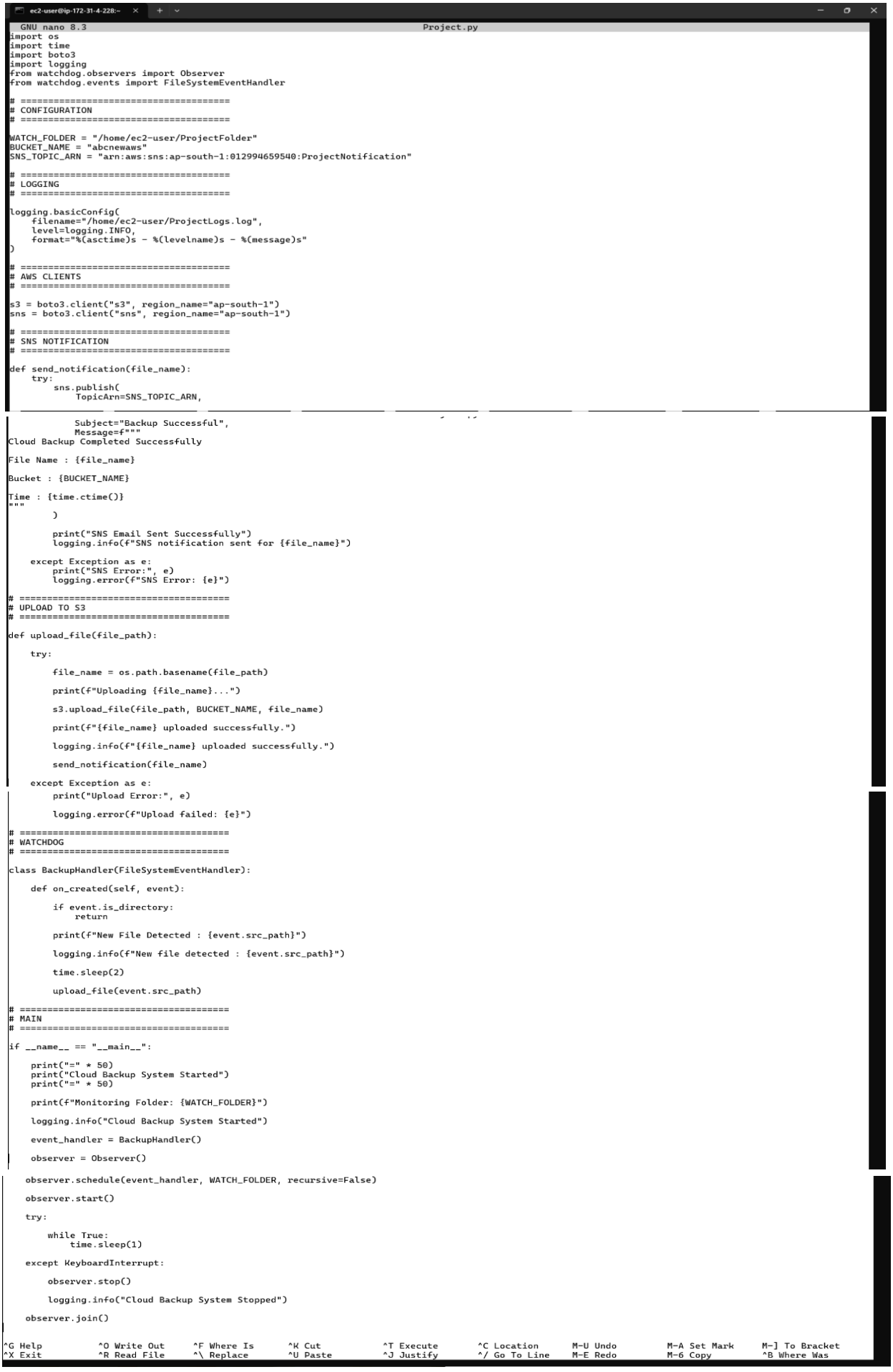


CloudWatch Log Created





CMD Window



```
ec2-user@ip-172-31-4-228:~
  GNU nano 8.3                                                    Project.py
import os
import time
import boto3
import logging
from watchdog.observers import Observer
from watchdog.events import FileSystemEventHandler

# ======================================
# CONFIGURATION
# ======================================

WATCH_FOLDER = "/home/ec2-user/ProjectFolder"
BUCKET_NAME = "abcnewaws"
SNS_TOPIC_ARN = "arn:aws:sns:ap-south-1:012994659540:ProjectNotification"

# ======================================
# LOGGING
# ======================================

logging.basicConfig(
    filename="/home/ec2-user/ProjectLogs.log",
    level=logging.INFO,
    format="%(asctime)s - %(levelname)s - %(message)s"
)

# ======================================
# AWS CLIENTS
# ======================================

s3 = boto3.client("s3", region_name="ap-south-1")
sns = boto3.client("sns", region_name="ap-south-1")

# ======================================
# SNS NOTIFICATION
# ======================================

def send_notification(file_name):
    try:
        sns.publish(
            TopicArn=SNS_TOPIC_ARN,
            Subject="Backup Successful",
            Message=f"""
Cloud Backup Completed Successfully

File Name : {file_name}

Bucket : {BUCKET_NAME}

Time : {time.ctime()}
"""
        )

        print("SNS Email Sent Successfully")
        logging.info(f"SNS notification sent for {file_name}")

    except Exception as e:
        print("SNS Error:", e)
        logging.error(f"SNS Error: {e}")

# ======================================
# UPLOAD TO S3
# ======================================

def upload_file(file_path):

    try:

        file_name = os.path.basename(file_path)

        print(f"Uploading {file_name}...")

        s3.upload_file(file_path, BUCKET_NAME, file_name)

        print(f"{file_name} uploaded successfully.")

        logging.info(f"{file_name} uploaded successfully.")

        send_notification(file_name)

    except Exception as e:
        print("Upload Error:", e)

        logging.error(f"Upload failed: {e}")

# ======================================
# WATCHDOG
# ======================================

class BackupHandler(FileSystemEventHandler):

    def on_created(self, event):

        if event.is_directory:
            return

        print(f"New File Detected : {event.src_path}")

        logging.info(f"New file detected : {event.src_path}")

        time.sleep(2)

        upload_file(event.src_path)

# ======================================
# MAIN
# ======================================

if __name__ == "__main__":

    print("=" * 50)
    print("Cloud Backup System Started")
    print("=" * 50)

    print(f"Monitoring Folder: {WATCH_FOLDER}")

    logging.info("Cloud Backup System Started")

    event_handler = BackupHandler()

    observer = Observer()

    observer.schedule(event_handler, WATCH_FOLDER, recursive=False)

    observer.start()

    try:

        while True:
            time.sleep(1)

    except KeyboardInterrupt:

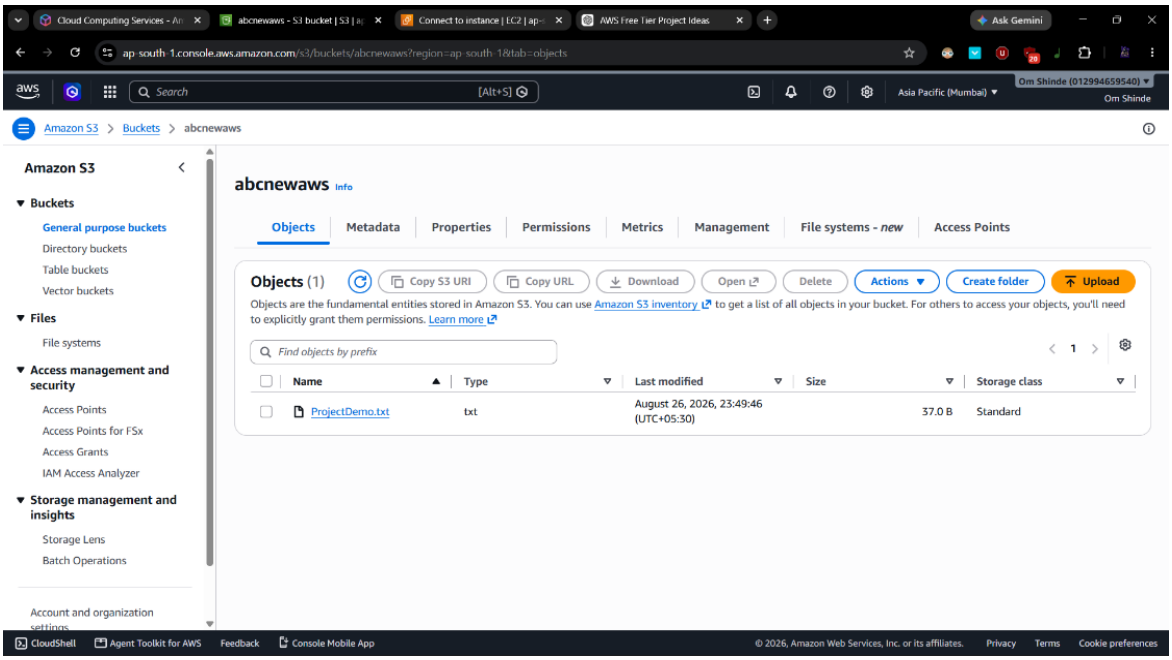
        observer.stop()

        logging.info("Cloud Backup System Stopped")

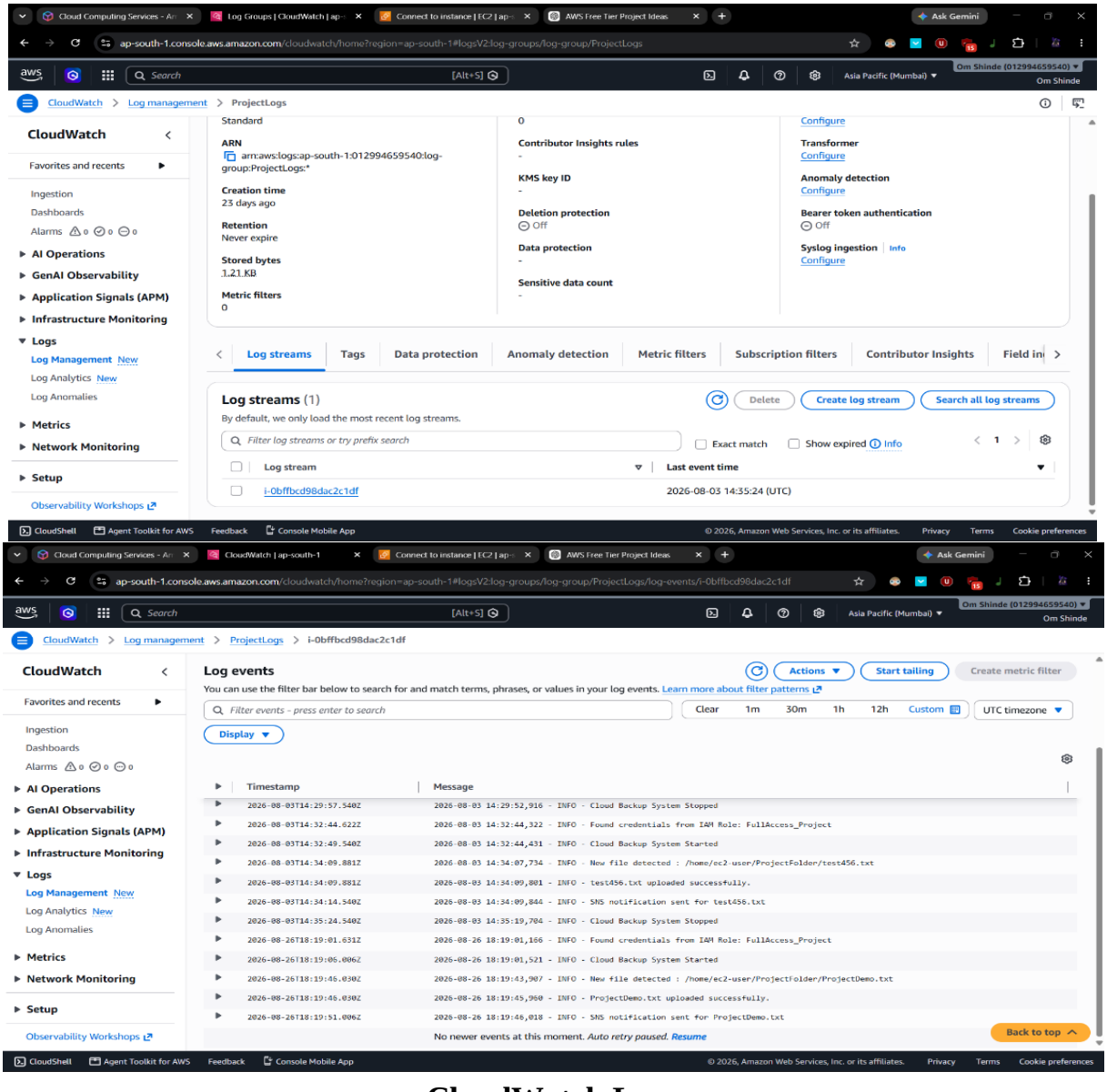
    observer.join()

^G Help          ^O Write Out     ^F Where Is      ^K Cut           ^T Execute       ^C Location      M-U Undo         M-A Set Mark     M-] To Bracket
^X Exit          ^R Read File     ^\ Replace       ^U Paste         ^J Justify       ^/ Go To Line    M-E Redo         M-6 Copy         ^B Where Was
```

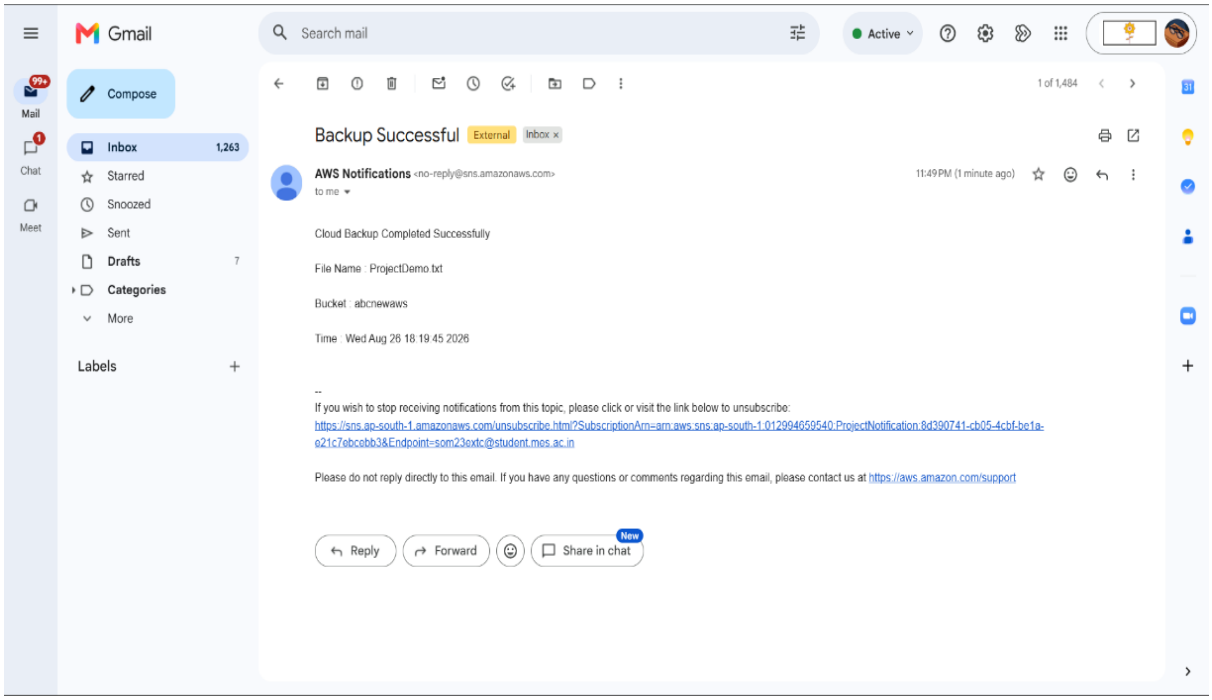
Python Code



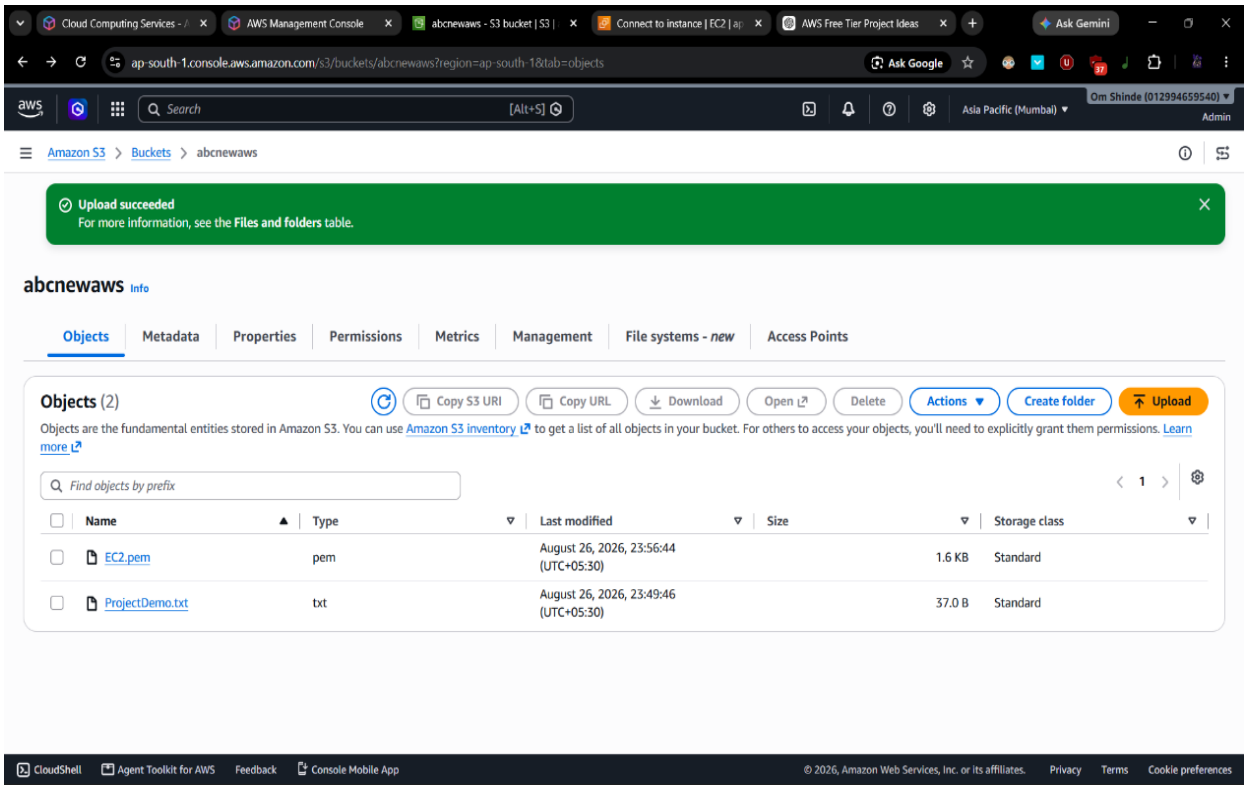
File Uploaded in S3 via Python Script



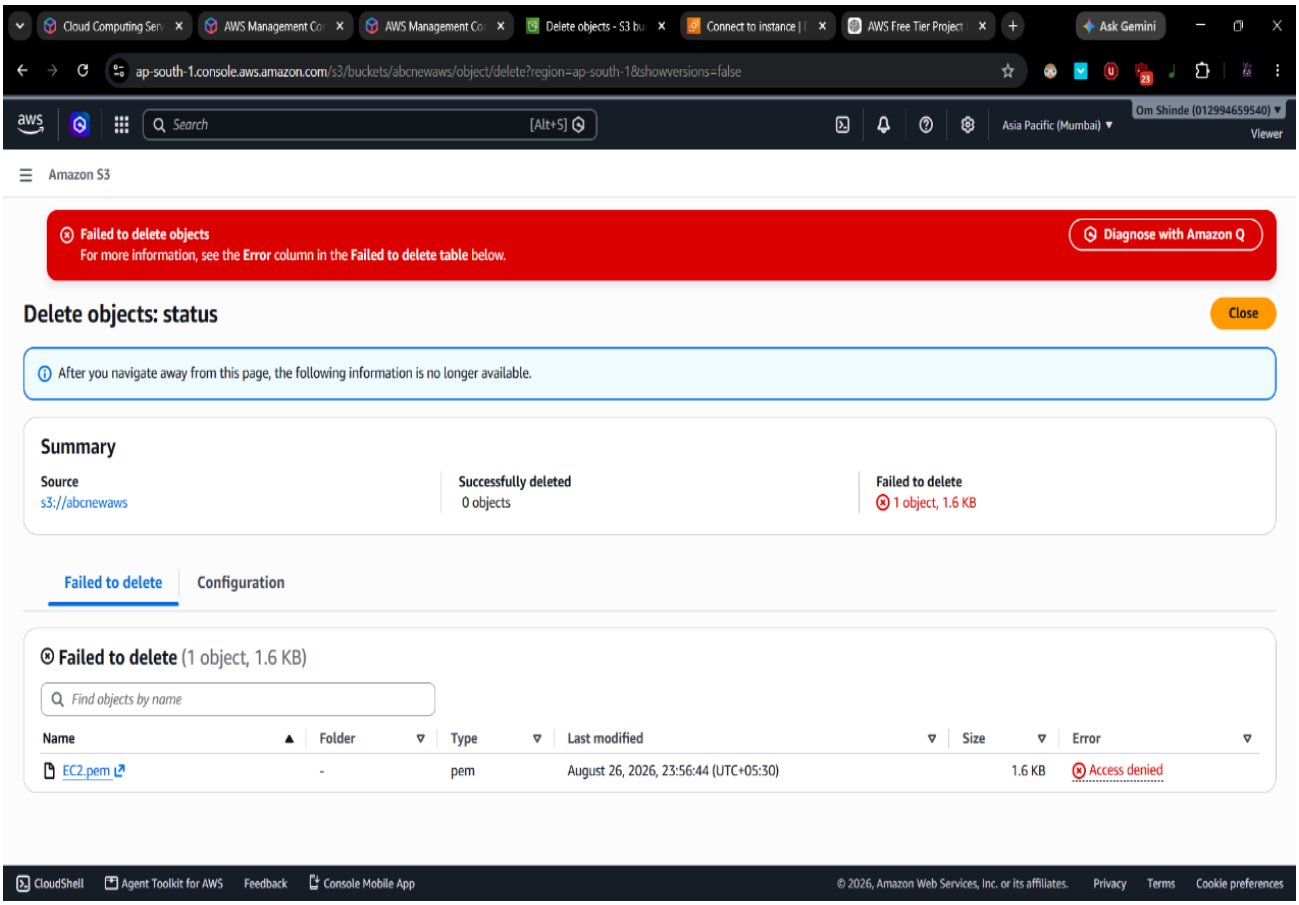
CloudWatch Logs



SNS Email Notification

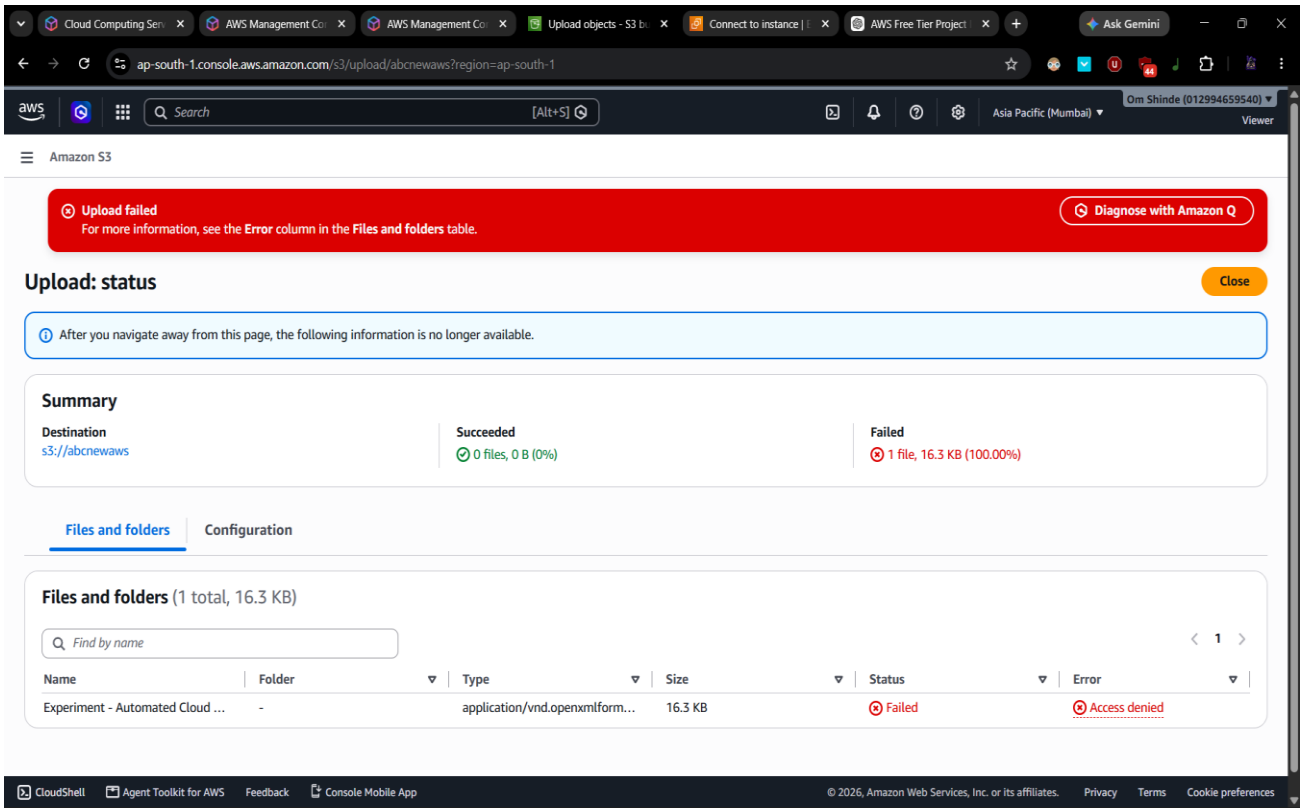


IAM User-Admin(Full S3 Access)



IAM User-Viewer(Only View Access)

Can’t delete objects from S3



IAM User-Viewer(Only View Access)

Can’t upload objects from S3

Result

On creating a new file inside ProjectFolder, the file was automatically detected by Watchdog, uploaded successfully to the S3 bucket 'abcnewaws', and a confirmation email was received via the SNS topic 'ProjectNotification'. All activities were recorded in ProjectLogs.log and were visible in real time under the CloudWatch log group 'ProjectLogs' in the AWS Console.

Conclusion

The Automated Cloud Backup and Monitoring System was successfully implemented by integrating Amazon EC2, S3, SNS, CloudWatch, and IAM with a Python application built using Watchdog and Boto3. The system eliminates manual backup effort, provides instant email confirmation of successful backups, and enables centralized, secure monitoring of application activity, thereby demonstrating a practical, automated, and access-controlled cloud solution.